Fakturama Image-to-Cash Automation: Design

Ali Abuelhana · Nov 3, 2026 · condensed from the Part 1 design doc

1. Goal and principles

One order image in; out: a saved, verified **Order** and an **Invoice** created from it as a follow-up document and correctly marked paid. Also out: the master data they need (Debtor, Payment Method, VAT, Products) and a run report. Delivery, Correction and Dunning documents are out of scope (task 5.7).

1. **Order first.** Open the New Order before anything else. Use its own Debtor and Product selectors as the existence checks.
2. **Create only on an exact miss**, save once, then return to the same open Order.
3. **Verify every step.** Read each value back from the UI before moving on.
4. **Stop rather than guess.** An ambiguous or conflicting match stops the run for manual review; the closest match is never taken.
5. **Find controls by meaning, not position.** No stored screen coordinates.

2. Architecture

```
image -> extract (vision LLM -> JSON) -> OCR cross-check -> normalize -> arithmetic checks
                                      nothing touches Fakturama before this passes
     -> flow: 1 Order -> 2 Debtor -> 3 Products -> 4 save Order -> 5 linked Invoice
              each stage ends with a verification gate
     -> UI driver (locators.yaml: UIA, label-relative, OCR) <--> Fakturama
     -> run report (JSON + Markdown, screenshots, manual-review file)
```

3. Image extraction

- **A. Vision LLM -> strict schema.** The model transcribes the image with structured output. It copies values verbatim and never calculates.
- **B. OCR cross-check.** Every critical value (reference, dates, SKUs, prices, totals, ZIPs) must also appear in a local OCR of the image. A missing value stops the run as a possible hallucination.
- **C. Normalize.** ISO dates, exact decimals, percentages as numbers, the payment method mapped to a fixed list, the contact name split into first and last name. Ambiguous input (`03/04/2026`, `1,234`) is an error, not a guess.
- **D. Validate.** Line = qty × unit net × (1 – discount); Net = Σ lines; VAT = Σ line × rate; Gross = Net + VAT. Required fields; PAID needs a payment date. Sample: 450.00 + 120.00 = 570.00, VAT 108.30, total 678.30.

Schema: order (reference, date, customer ID, currency), debtor (company, names, alias, e-mail, phone), billing and delivery address, payment (method, status, date), items (SKU, description, qty, unit net, discount, VAT, line net), totals.

4. Finding controls

- **UI Automation first** (pywinauto, UIA backend): name + control type, scoped to the main window, an editor tab, a dialog or a menu.
- **Label-relative lookup** for fields without a name: the editable control after its label (`Cust.Ref.`, `Date`, ...), resolved fresh on every run.
- **OCR on a capture of one control** where UIA sees nothing. Fakturama draws its lists itself, so the list rows are read this way, assigned to columns by the table's grid lines, and clicked where OCR found them.
- **One locator registry** (`config/locators.yaml`). If Fakturama's UI changes, only this file changes.
- **Safe interaction.** Wait for a state, not for time. Wait for a list to stop changing after a search. Set a whole search text in one step, because Fakturama's selectors pick a row by themselves when one is left. Read back after every write. Use the "select existing" icons, never the green "+".

5. Flow (task step numbers in brackets)

Stage	Actions	Gate
1 Open the New Order	Toolbar Order; keep the proposed No.; set Date, Cust.Ref., Net, With VAT [1.3–1.8]	Order tab open, values read back
2 Debtor	Select the address: search the company and compare Company, First Name, Name, ZIP and City [2.1–2.3]. One exact row: select it [2.4]. Several or conflicting: stop. None: create it: payment method found or created first, then New Contact with the proposed ID, names, main address with the Invoice role and a second address with the Delivery role if the addresses differ, Miscellaneous (alias, discount 0, Net, payment method). Save once and reselect it in the Order [2.5–2.12]	Both Order addresses equal the image [2.13]
3 Products (per item, in source order)	Select a product: search the exact SKU [3.1–3.3]. None: create or reuse VAT 19% (name, value, code S) [3.4–3.6], create the product with Price (gross) = unit net × (1 + VAT) (250.00 × 1.19 = 297.50, no line discount), cost 0.00, stock 0.00; save once and reselect [3.7–3.12]. Set Qty., U.Price and Discount on the line [3.13–3.15]	Total Net and VAT equal the lines done so far (450.00, then 570.00) [3.16–3.17]
4 Save the Order	Re-check addresses and every line, discount 0%, Free of shipping costs 0.00, totals [4.1–4.3]; Save once [4.4]; Data > Documents [4.5]; follow-up Invoice from the Order, not the toolbar [4.6–4.7]	One open Order row with number, Date, Cust.Ref. and Total
5 Linked Invoice	Keep the proposed No. and dates; check copied Cust.Ref., addresses, Order Date, VAT mode, lines, totals [5.1]; set the payment method [5.2]; if PAID tick paid, payment date, Value = Total, otherwise leave it clear [5.3]; Save once [5.4]	Documents: Invoice state and Total, Order still open; saved payment fields [5.5–5.6]

6. Verification and failure handling

Three levels: **field** (read back after each write), **step** (a selection is confirmed by its effect, e.g. the addresses it fills in), **record** (Data > Documents after each save).

Situation	Action
Extraction check fails	Stop before Fakturama is touched
Control not found in time	Stop with the screen's state
Read-back mismatch	Stop at that field
Ambiguous or conflicting match	Stop for manual review
Unexpected dialog	Log its text, cancel it, stop
Fakturama totals differ from the image	Stop before saving the Order

Each stop writes `manual_review.json`: stage, reason, failed checks, editors left open (unsaved ones marked) and a screenshot. A changed editor is only ever closed by discarding it. **Reruns** repeat the exact-match checks, so master data saved by a stopped run is reused, not duplicated.

7. Trade-offs

Decision	Why	Cost
UIA first, OCR where UIA sees nothing	Fast, exact, independent of window size	OCR of lists is the slowest and most fragile part

Vision LLM for extraction	Any layout, photos, tables	Cost, network, can hallucinate, hence checks B–D
Exact match only	No wrong customer or product is booked	More manual reviews; near-duplicates are stopped, not merged
Drive the UI, not the database	Required; Fakturama applies its own numbering, totals and links	Minutes per order instead of seconds
Read back after every write	Errors surface at the step that caused them	About twice the UI calls

Rejected: a pure screenshot-and-click LLM agent (slow, costly, not repeatable) and image templates for buttons (break with themes, DPI and language).

8. As built

Python 3.11, pywinauto, Windows OCR (Apple Vision for development on macOS), pydantic, PyYAML; modules `extract/`, `ui/`, `flow/`, `report/`. The whole flow is built and was recorded live from an empty Fakturama 2.2.0 workspace: 205 checks, none failed, about 3 minutes.

Changes made while building:

- the payment method is resolved before New Contact, because Fakturama 2.2.0 fills the contact's Payment list when the editor opens;
- a read-back mismatch stops at once instead of being retried;
- OCR is the main reader of Fakturama's lists, not only a fallback;
- the planned duplicate-order guard (search Documents for the Cust.Ref. first) is not built yet.